UDP Chat Client and Server — Lab Exam Algorithms

UDP Chat Client — Algorithm

1. Start the client program.

2. Create a `BufferedReader` object to read messages entered by the user from the keyboard.

```
BufferedReader br =
    new BufferedReader(
        new InputStreamReader(System.in));
```

3. Create a `DatagramSocket` using port `8050` for UDP communication.

```
DatagramSocket client = new DatagramSocket(8050);
```

4. Get the local host address of the system.

```
InetAddress address = InetAddress.getLocalHost();
```

5. Display a message instructing the user to press Enter without text to quit.

```
System.out.println("Press Enter without text to quit");
```

6. Repeat the following steps inside a loop and read a message from the user.

```
while (true)
{
    System.out.print("Enter text for server : ");
    String msg = br.readLine();
```

7. Convert the message into a byte array.

```
byte[] data = msg.getBytes();
```

8. Create a `DatagramPacket` containing the message and specify the server address and port `8040`.

```
DatagramPacket packet =
    new DatagramPacket(
        data, data.length, address, 8040);
```

9. Send the packet to the server.

```
client.send(packet);
```

10. Check whether the message is empty. If it is empty, terminate the loop.

```
if (msg.trim().length() == 0)
    break;
```

11. Create a byte array and `DatagramPacket` to receive the reply from the server.

```
byte[] receiveData = new byte[1024];

DatagramPacket receivePacket =
    new DatagramPacket(receiveData, receiveData.length);
```

12. Receive the reply from the server.

```
client.receive(receivePacket);
```

13. Convert the received data into a string and display the server's reply.

```
String reply = new String(
    receivePacket.getData(), 0, receivePacket.getLength());

System.out.println("From Server <<< " + reply);
```

14. Repeat the communication until the user enters an empty message.

15. Close the client socket.

```
client.close();
```

16. Stop the program.

UDP Chat Server — Algorithm

1. Start the server program.

2. Create a `DatagramSocket` using port `8040` to receive messages from the client.

```
DatagramSocket server = new DatagramSocket(8040);
```

3. Create a `BufferedReader` object to read reply messages entered by the server user.

```
BufferedReader br =
    new BufferedReader(
        new InputStreamReader(System.in));
```

4. Display a message indicating that the server is ready.

```
System.out.println("Server Ready");
```

5. Repeat the following steps inside a loop and create a byte array and `DatagramPacket` to receive a message from the client.

```
while (true)
{
    byte[] data = new byte[1024];

    DatagramPacket packet =
        new DatagramPacket(data, data.length);
```

6. Receive the packet from the client.

```
server.receive(packet);
```

7. Convert the received data into a string.

```
String msg = new String(
    packet.getData(), 0, packet.getLength());
```

8. Check whether the received message is empty. If it is empty, terminate the loop.

```
if (msg.trim().length() == 0)
    break;
```

9. Display the message received from the client.

```
System.out.println("From Client <<< " + msg);
```

10. Read a reply message from the server user.

```
System.out.print("Msg to Client : ");
String reply = br.readLine();
```

11. Convert the reply into a byte array.

```
byte[] sendData = reply.getBytes();
```

12. Create a `DatagramPacket` containing the reply and use the client's address and port `8050`.

```
DatagramPacket sendPacket =
    new DatagramPacket(
        sendData, sendData.length,
        packet.getAddress(), 8050);
```

13. Send the reply packet to the client.

```
server.send(sendPacket);
```

14. Repeat the communication until the client sends an empty message.

15. Display that the client has quit.

```
System.out.println("Client Quits");
```

16. Close the server socket.

```
server.close();
```

17. Stop the program.